Code for Project

Jason, Patricia, Christopher

2025-05-12

#Importing the libraries

```
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##     filter, lag

## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(tidyr)
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v forcats   1.0.0     v readr     2.1.2
## v lubridate 1.9.2     v stringr   1.5.1
## v purrr     1.0.4     v tibble    3.2.1

## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(glmnet)
```

```
## Loading required package: Matrix
##
## Attaching package: 'Matrix'
##
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
##
## Loaded glmnet 4.1-8
```

```
library(randomForest)
```

```
## randomForest 4.7-1.1
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##     combine
##
## The following object is masked from 'package:ggplot2':
##
##     margin
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##     lift
```

```
library(forcats)
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
##
## Attaching package: 'pROC'
##
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:randomForest':
##
##     combine
##
## The following object is masked from 'package:dplyr':
##
##     combine
```

```r
library(reshape2)
```

```
##
## Attaching package: 'reshape2'
##
## The following object is masked from 'package:tidyr':
##
##     smiths
```

#Importing data

```
## 'data.frame':    1000 obs. of  15 variables:
##  $ age                          : int  23 20 21 23 19 24 21 21 23 18 ...
##  $ gender                       : chr  "Female" "Female" "Male" "Female" ...
##  $ study_hours_per_day          : num  0 6.9 1.4 1 5 7.2 5.6 4.3 4.4 4.8 ...
##  $ social_media_hours           : num  1.2 2.8 3.1 3.9 4.4 1.3 1.5 1 2.2 3.1 ...
##  $ netflix_hours                : num  1.1 2.3 1.3 1 0.5 0 1.4 2 1.7 1.3 ...
##  $ part_time_job                : chr  "No" "No" "No" "No" ...
##  $ attendance_percentage        : num  85 97.3 94.8 71 90.9 82.9 85.8 77.7 100 95.4 ...
##  $ sleep_hours                  : num  8 4.6 8 9.2 4.9 7.4 6.5 4.6 7.1 7.5 ...
##  $ diet_quality                 : chr  "Fair" "Good" "Poor" "Poor" ...
##  $ exercise_frequency           : int  6 6 1 4 3 1 2 0 3 5 ...
##  $ parental_education_level     : chr  "Master" "High School" "High School" "Master" ...
##  $ internet_quality             : chr  "Average" "Average" "Poor" "Good" ...
##  $ mental_health_rating         : int  8 8 1 1 1 4 4 8 1 10 ...
##  $ extracurricular_participation: chr  "Yes" "No" "No" "Yes" ...
##  $ exam_score                   : num  56.2 100 34.3 26.8 66.4 100 89.8 72.6 78.9 100 ...
```

```
## [1] 0
```

```
##
## Call:
## lm(formula = exam_score ~ ., data = student)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.2097  -3.4768   0.0789   3.4456  15.5955
##
## Coefficients:
##                                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)                      7.17718    2.50263   2.868  0.00422 **
## age                             -0.01209    0.07365  -0.164  0.86959
## genderMale                       0.14621    0.34805   0.420  0.67452
## genderOther                      0.79264    0.86559   0.916  0.36003
## study_hours_per_day              9.57454    0.11577  82.700  < 2e-16 ***
## social_media_hours              -2.60220    0.14489 -17.960  < 2e-16 ***
## netflix_hours                   -2.28156    0.15777 -14.462  < 2e-16 ***
## part_time_jobYes                 0.21121    0.41410   0.510  0.61013
## attendance_percentage            0.14339    0.01821   7.876 8.94e-15 ***
## sleep_hours                      1.99234    0.13849  14.386  < 2e-16 ***
## diet_qualityGood                -0.68310    0.37777  -1.808  0.07088 .
## diet_qualityPoor                -0.27224    0.47306  -0.575  0.56510
```

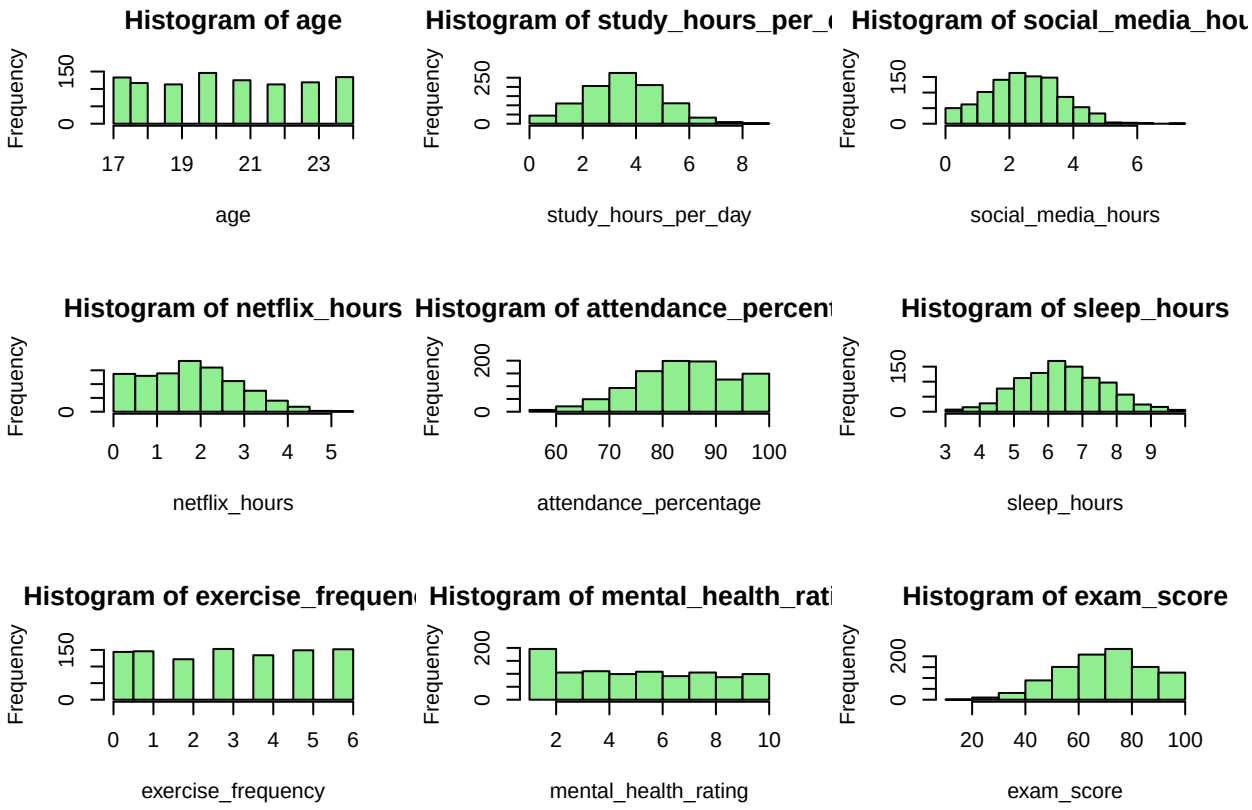
```
## exercise_frequency                   1.45004    0.08397  17.268  < 2e-16 ***
## parental_education_levelHigh School -0.15995    0.39605  -0.404  0.68639
## parental_education_levelMaster      -0.41093    0.50782  -0.809  0.41860
## parental_education_levelNone        -0.70202    0.63313  -1.109  0.26778
## internet_qualityGood                -0.47289    0.37292  -1.268  0.20507
## internet_qualityPoor                -0.08167    0.50324  -0.162  0.87111
## mental_health_rating                 1.94417    0.06003  32.386  < 2e-16 ***
## extracurricular_participationYes    -0.01412    0.36426  -0.039  0.96909
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 5.342 on 980 degrees of freedom
## Multiple R-squared:  0.9018, Adjusted R-squared:  0.8999
## F-statistic: 473.9 on 19 and 980 DF,  p-value: < 2.2e-16
```

#EDA Of data

```
colSums(is.na(student))
```

```
##                            age                         gender
##                              0                              0
##            study_hours_per_day              social_media_hours
##                              0                              0
##                  netflix_hours                   part_time_job
##                              0                              0
##          attendance_percentage                    sleep_hours
##                              0                              0
##                   diet_quality              exercise_frequency
##                              0                              0
##       parental_education_level                internet_quality
##                              0                              0
##           mental_health_rating extracurricular_participation
##                              0                              0
##                     exam_score
##                              0
```

```
numeric_vars <- student %>% select_if(is.numeric)

#Distributions for all numeric values in the form of histogram.
par(mfrow = c(3, 3))
for (var in names(numeric_vars)) {
  hist(numeric_vars[[var]], main = paste("Histogram of", var),
       xlab = var,
       col = "lightgreen")}
```



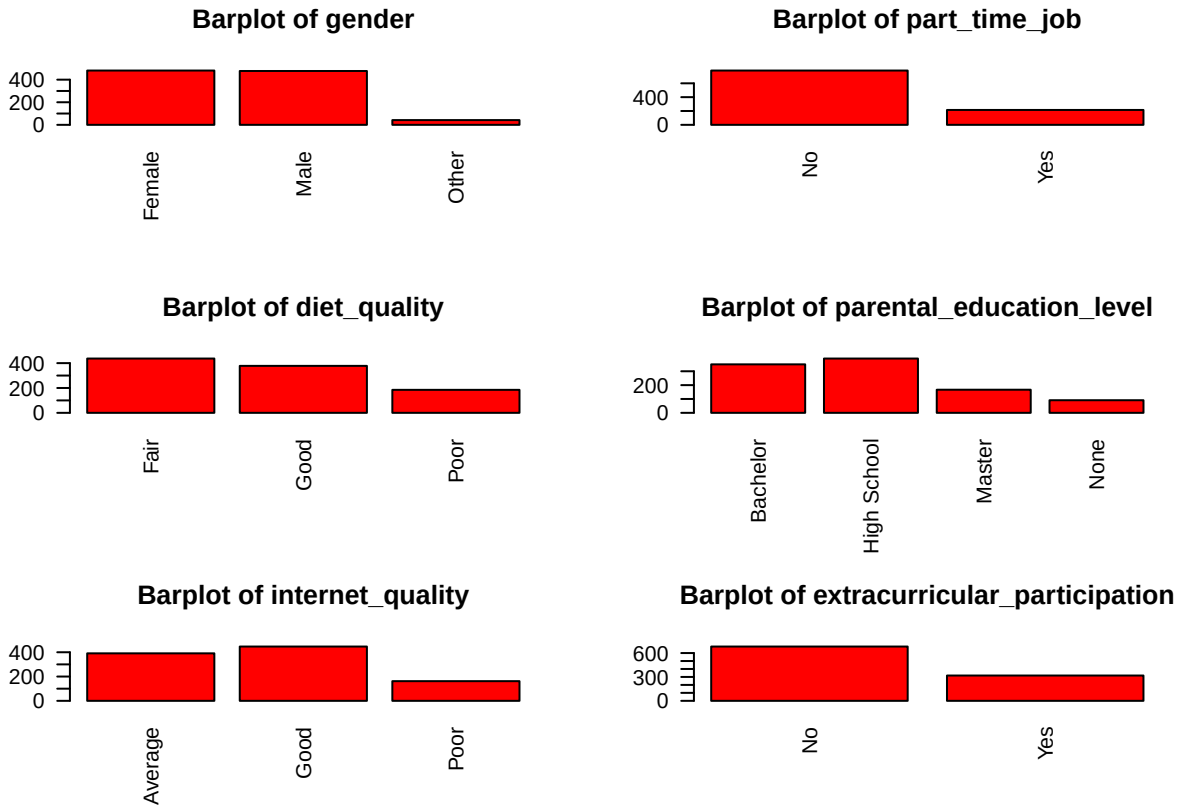
#Factoring all character datatypes

```
student_factor <- student %>%
  mutate(across(where(is.character), as.factor))

str(student_factor)
```

```
## 'data.frame':    1000 obs. of  15 variables:
##  $ age                          : int  23 20 21 23 19 24 21 21 23 18 ...
##  $ gender                       : Factor w/ 3 levels "Female","Male",..: 1 1 2 1 1 2 1 1 1 1 ...
##  $ study_hours_per_day          : num  0 6.9 1.4 1 5 7.2 5.6 4.3 4.4 4.8 ...
##  $ social_media_hours           : num  1.2 2.8 3.1 3.9 4.4 1.3 1.5 1 2.2 3.1 ...
##  $ netflix_hours                : num  1.1 2.3 1.3 1 0.5 0 1.4 2 1.7 1.3 ...
##  $ part_time_job                : Factor w/ 2 levels "No","Yes": 1 1 1 1 1 1 2 2 1 1 ...
##  $ attendance_percentage        : num  85 97.3 94.8 71 90.9 82.9 85.8 77.7 100 95.4 ...
##  $ sleep_hours                  : num  8 4.6 8 9.2 4.9 7.4 6.5 4.6 7.1 7.5 ...
##  $ diet_quality                 : Factor w/ 3 levels "Fair","Good",..: 1 2 3 3 1 1 2 1 2 2 ...
##  $ exercise_frequency           : int  6 6 1 4 3 1 2 0 3 5 ...
##  $ parental_education_level     : Factor w/ 4 levels "Bachelor","High School",..: 3 2 2 3 3 3 3 1 1 1
##  $ internet_quality             : Factor w/ 3 levels "Average","Good",..: 1 1 3 2 2 1 3 1 2 2 ...
##  $ mental_health_rating         : int  8 8 1 1 1 4 4 8 1 10 ...
##  $ extracurricular_participation: Factor w/ 2 levels "No","Yes": 2 1 1 2 1 1 1 1 1 2 ...
##  $ exam_score                   : num  56.2 100 34.3 26.8 66.4 100 89.8 72.6 78.9 100 ...
```

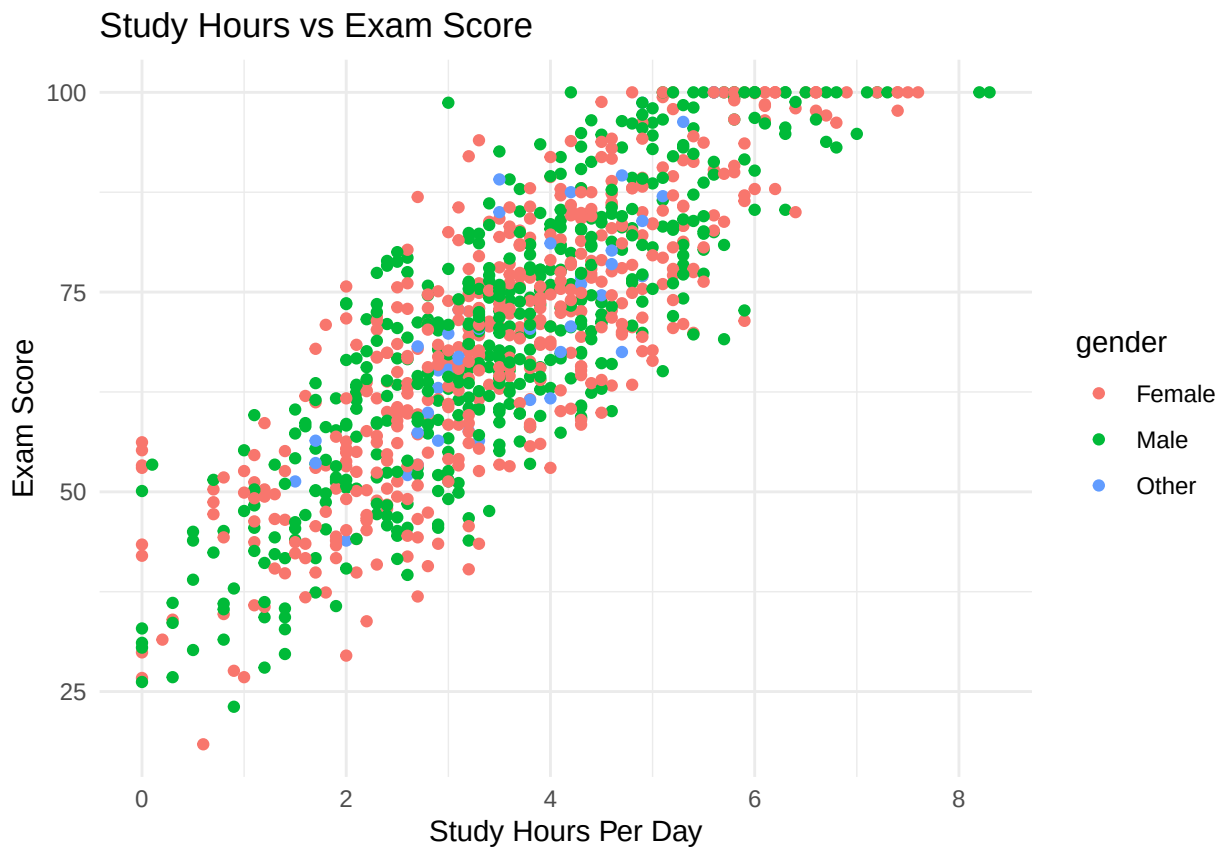
#Visualizing Distributions of Factor Variables.

```r
factor_vars <- student_factor %>%
  select(where(is.factor))

par(mfrow = c(3, 2))

for (var in names(factor_vars)) {
  barplot(table(factor_vars[[var]]),
          main = paste("Barplot of", var),
          col = "red",
          las = 2)
}
```



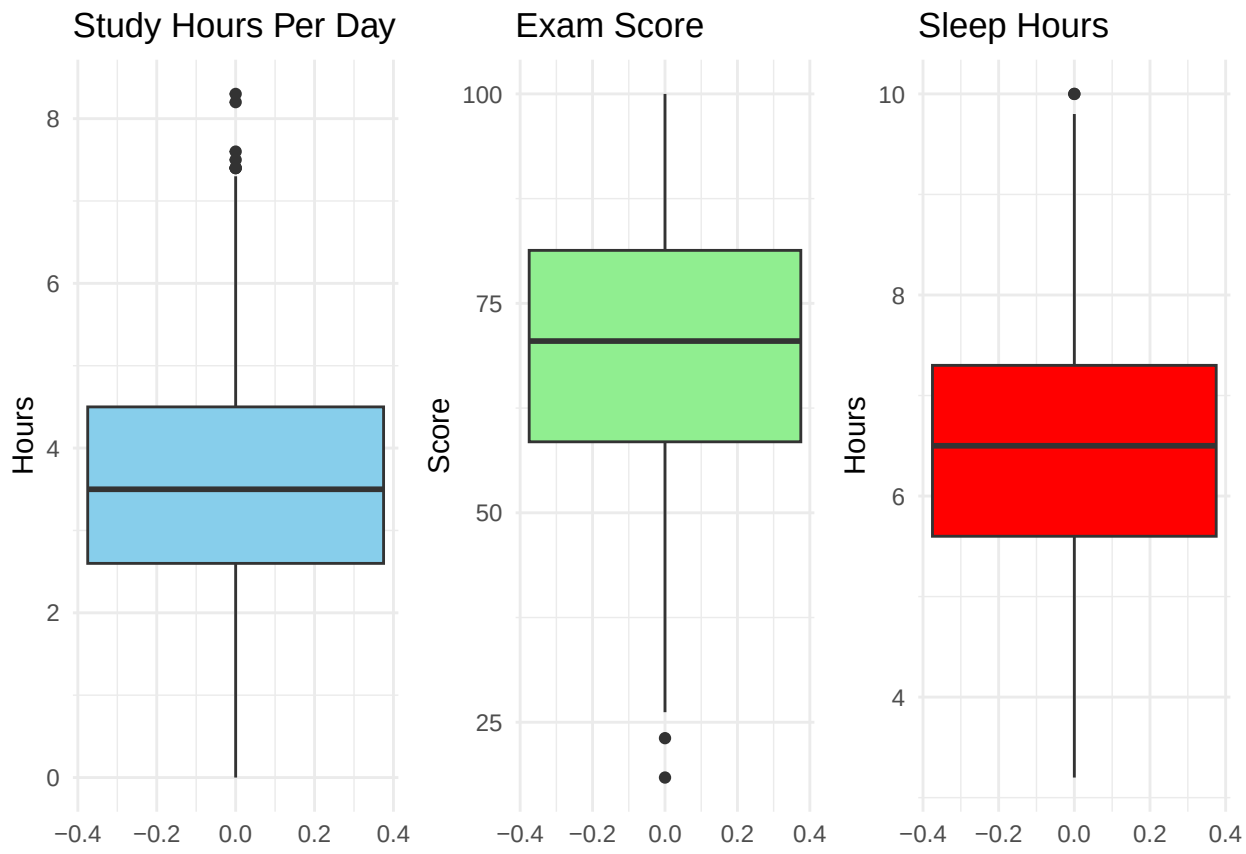
#Scatter plot for study hours daily vs. exam scores

```r
ggplot(data, aes(x = study_hours_per_day, y = exam_score, color = gender)) +
  geom_point() +
  labs(title = "Study Hours vs Exam Score",
       x = "Study Hours Per Day",
       y = "Exam Score") +
  theme_minimal()
```



#Creating boxplots for study time, exam score, and sleep in hours

```
study_box <- ggplot(student_factor, aes(y = study_hours_per_day)) +
  geom_boxplot(fill = "skyblue") +
  labs(title = "Study Hours Per Day", y = "Hours") +
  theme_minimal()

score_box <- ggplot(student_factor, aes(y = exam_score)) +
  geom_boxplot(fill = "lightgreen") +
  labs(title = "Exam Score", y = "Score") +
  theme_minimal()


sleep_box <- ggplot(student_factor, aes(y = sleep_hours)) +
  geom_boxplot(fill = "red") +
  labs(title = "Sleep Hours", y = "Hours") +
  theme_minimal()

grid.arrange(study_box, score_box, sleep_box, ncol = 3)
```



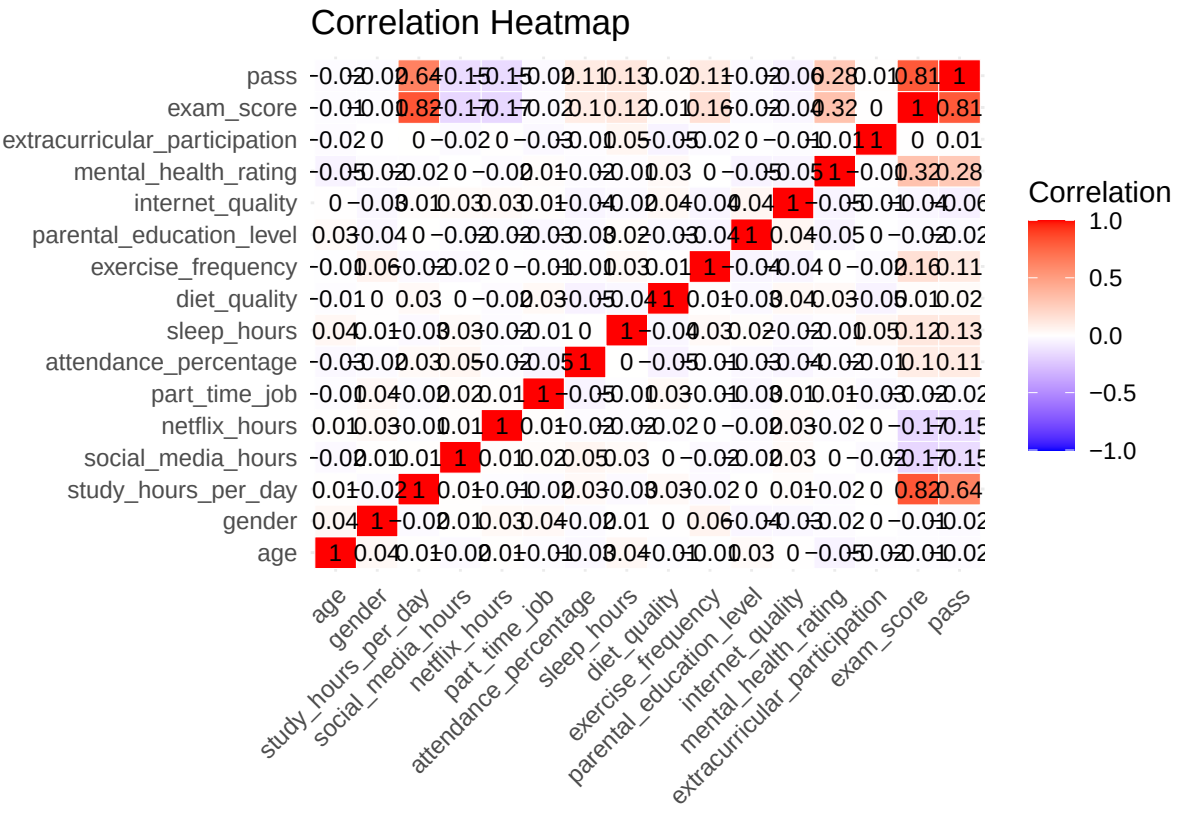
#Converting all factorized values into numeric

```
corr_data <- student_factor

corr_data <- corr_data %>%
  mutate(
    gender = as.numeric(factor(gender)),
    part_time_job = as.numeric(factor(part_time_job)),
    diet_quality = as.numeric(factor(diet_quality, levels = c("Poor", "Fair", "Good"), ordered = TRUE))
    internet_quality = as.numeric(factor(internet_quality, levels = c("Poor", "Average", "Good"), ordere
    extracurricular_participation = as.numeric(factor(extracurricular_participation)),
    parental_education_level = as.numeric(factor(parental_education_level, levels = c("High School", "Ba
    pass = ifelse(exam_score >= 70, 1, 0)
  )
```

#Creating correlation heatmap

```
cor_matrix <- cor(corr_data, use = "complete.obs")
cor_melt <- melt(cor_matrix)

ggplot(cor_melt, aes(Var1, Var2, fill = value)) +
  geom_tile(color = "white") +
  geom_text(aes(label = round(value, 2)), size = 3, color = "black") +
  scale_fill_gradient2(low = "blue", high = "red", mid = "white",
                       midpoint = 0, limit = c(-1, 1), space = "Lab",
                       name = "Correlation") +
```

```
  theme_minimal() +
  labs(title = "Correlation Heatmap", x = "", y = "") +
  theme(axis.text.x = element_text(angle = 45, vjust = 1, hjust = 1))
```



#Can we predict student scores based on lifestyle, study habits, attendance, and sleep hours?

```
#Engineering Feature
student_factor$total_screentime <- student_factor$social_media_hours + student_factor$netflix_hours

#Subset the variables
student_subset <- student_factor %>%
  select(
         study_hours_per_day,
         attendance_percentage,
         sleep_hours,
         total_screentime,
         part_time_job,
         exam_score,
         exercise_frequency,
         diet_quality,
         extracurricular_participation
         )

#Splitting to Train and Testing

set.seed(1)
```

```r
train = sample(1:nrow(student_subset), nrow(student_subset)*.8)
test=(-train)



#Creating Linear Model

# 2. Train the model without cross-Validation
lm_model <- lm(exam_score ~ study_hours_per_day * sleep_hours + part_time_job +
                 total_screentime + attendance_percentage + exercise_frequency +
                 diet_quality + extracurricular_participation,
               data = student_subset[train, ])

lm_pred <- predict(lm_model, newdata = student_subset[test, ])

x=model.matrix(exam_score~., student)[,-1]

y=student$exam_score

y_test <- y[test]
```

#Summary and metrics (R^2 and RMSE)

```r
lm_rmse <- RMSE(lm_pred, y_test) #*updated to y_test instead of y.test
lm_r2 <- R2(lm_pred, y_test)
lm_mae <- mean(abs(lm_pred - y_test))


summary(lm_model)
```

```
##
## Call:
## lm(formula = exam_score ~ study_hours_per_day * sleep_hours +
##     part_time_job + total_screentime + attendance_percentage +
##     exercise_frequency + diet_quality + extracurricular_participation,
##     data = student_subset[train, ])
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.4512  -5.3527  -0.1359   5.2349  24.5413
##
## Coefficients:
##                                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)                     17.16357    4.67127   3.674 0.000255 ***
## study_hours_per_day              9.61948    0.99989   9.621  < 2e-16 ***
## sleep_hours                      2.01664    0.57191   3.526 0.000446 ***
## part_time_jobYes                 0.45176    0.67242   0.672 0.501879
## total_screentime                -2.50216    0.17562 -14.248  < 2e-16 ***
## attendance_percentage            0.13972    0.02908   4.804 1.86e-06 ***
## exercise_frequency               1.50927    0.13591  11.105  < 2e-16 ***
## diet_qualityGood                -0.43415    0.61519  -0.706 0.480573
## diet_qualityPoor                -0.07637    0.76466  -0.100 0.920467
## extracurricular_participationYes  0.47353    0.59172   0.800 0.423807
```

```
## study_hours_per_day:sleep_hours  -0.01946    0.15161  -0.128 0.897923
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.764 on 789 degrees of freedom
## Multiple R-squared:  0.7935, Adjusted R-squared:  0.7908
## F-statistic: 303.1 on 10 and 789 DF,  p-value: < 2.2e-16
```

```
print(paste("R-squared of linear model:", round(lm_r2, 4)))
```

```
## [1] "R-squared of linear model: 0.788"
```

```
print(paste("RMSE of linear model:", round(lm_rmse, 4)))
```

```
## [1] "RMSE of linear model: 7.6124"
```

```
print(paste("MAE of linear model:", round(lm_mae, 4)))
```

```
## [1] "MAE of linear model: 6.2463"
```

2. Train the model with cross-Validation

```
cv_control <- trainControl(method = "cv", number = 10)

cv_model <- train(
  exam_score ~ study_hours_per_day * sleep_hours + part_time_job +
    total_screentime + attendance_percentage + exercise_frequency +
    diet_quality + extracurricular_participation, #update predictors######
  data = student_subset[train, ],
  method = "lm",
  trControl = cv_control
)

summary(cv_model)
```

```
##
## Call:
## lm(formula = .outcome ~ ., data = dat)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -22.4512  -5.3527  -0.1359   5.2349  24.5413
##
## Coefficients:
##                                 Estimate Std. Error t value Pr(>|t|)
## (Intercept)                     17.16357    4.67127   3.674 0.000255 ***
## study_hours_per_day              9.61948    0.99989   9.621  < 2e-16 ***
## sleep_hours                      2.01664    0.57191   3.526 0.000446 ***
```

```
## part_time_jobYes                   0.45176    0.67242   0.672 0.501879
## total_screentime                  -2.50216    0.17562 -14.248  < 2e-16 ***
## attendance_percentage              0.13972    0.02908   4.804 1.86e-06 ***
## exercise_frequency                 1.50927    0.13591  11.105  < 2e-16 ***
## diet_qualityGood                  -0.43415    0.61519  -0.706 0.480573
## diet_qualityPoor                  -0.07637    0.76466  -0.100 0.920467
## extracurricular_participationYes   0.47353    0.59172   0.800 0.423807
## `study_hours_per_day:sleep_hours` -0.01946    0.15161  -0.128 0.897923
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 7.764 on 789 degrees of freedom
## Multiple R-squared:  0.7935, Adjusted R-squared:  0.7908
## F-statistic: 303.1 on 10 and 789 DF,  p-value: < 2.2e-16
```

```
print(paste("Cross-validated Linear Model R-Squared:", round(cv_model$results$Rsquared, 4)))
```

```
## [1] "Cross-validated Linear Model R-Squared: 0.7886"
```

```
print(paste("Cross-validated Linear Model RMSE:", round(cv_model$results$RMSE, 4)))
```

```
## [1] "Cross-validated Linear Model RMSE: 7.7929"
```

```
print(paste("Cross-validated Linear Model MAE:", round(cv_model$results$MAE, 4)))
```

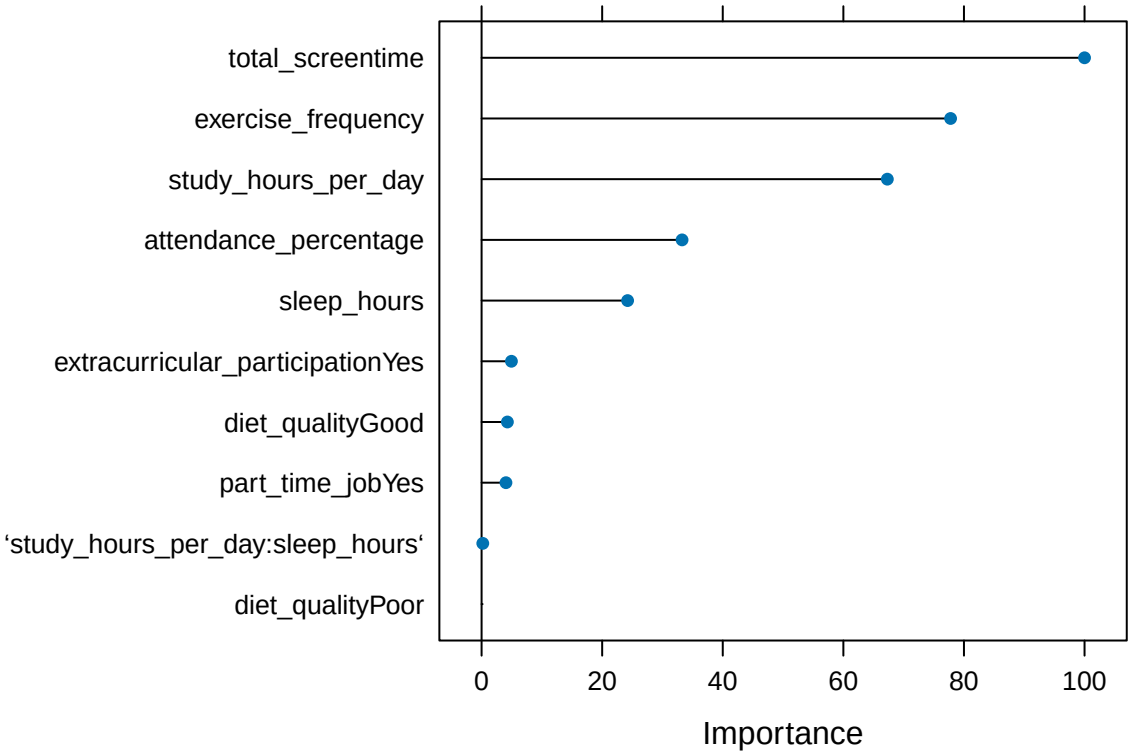
```
## [1] "Cross-validated Linear Model MAE: 6.3393"
```

4. Evaluate performance

```
# 4. Evaluate performance
lm_pred_cv <- predict(cv_model, newdata = student_subset[test, ]) #added prediction test########


# 4. Evaluate performance
lm_actuals_cv <- student_subset[test, ]$exam_score
lm_performance_cv <- postResample(pred = lm_pred_cv, obs = lm_actuals_cv)
print(lm_performance_cv)
```

```
##     RMSE Rsquared      MAE
## 7.612438 0.787951 6.246318
```

```
# 5. Variable Importance
plot(varImp(cv_model))
```

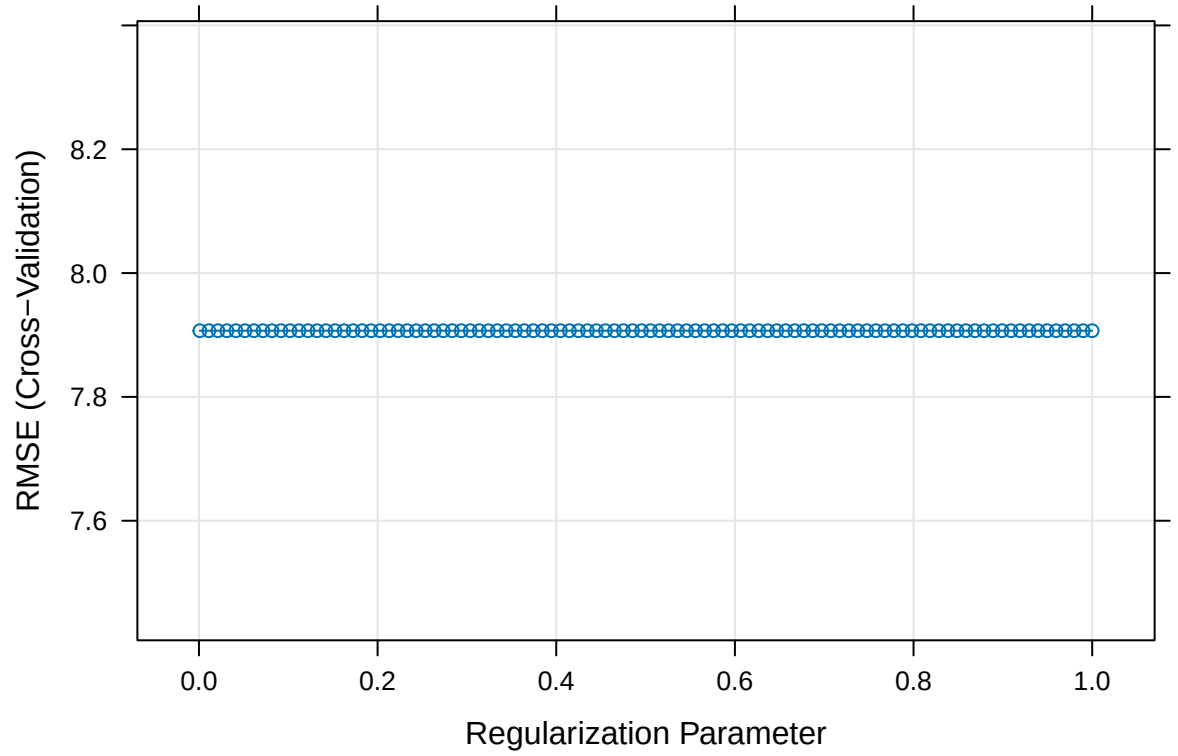


#Creating Ridge Model

```
ridge_grid <- expand.grid(
  alpha = 0,                    # alpha = 0 → Ridge
  lambda = seq(0.001, 1, length = 100)  # lambda values to test
)

# 2. Train the model
set.seed(22)
cv_ridge_model <- train(
  exam_score ~ study_hours_per_day * sleep_hours + part_time_job +
    total_screentime + attendance_percentage + exercise_frequency +
    diet_quality + extracurricular_participation,
  data = student_subset[train, ],
  method = "glmnet",
  trControl = cv_control,
  tuneGrid = ridge_grid
)

# Get the best lambda chosen by cross-validation
ridge_best_lambda <- cv_ridge_model$bestTune$lambda

# Get ridge model coefficients
ridge_coefs <- coef(cv_ridge_model$finalModel, s = ridge_best_lambda)
print(ridge_coefs)
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##                                        s1
## (Intercept)                     32.0730100
## study_hours_per_day              5.5999982
## sleep_hours                      0.1124347
## part_time_jobYes                 0.3276043
## total_screentime                -2.3141560
## attendance_percentage            0.1255451
## exercise_frequency               1.3694307
## diet_qualityGood                -0.3927673
## diet_qualityPoor                -0.2568226
## extracurricular_participationYes 0.4544068
## study_hours_per_day:sleep_hours  0.5280259
```

```
plot(cv_ridge_model)
```



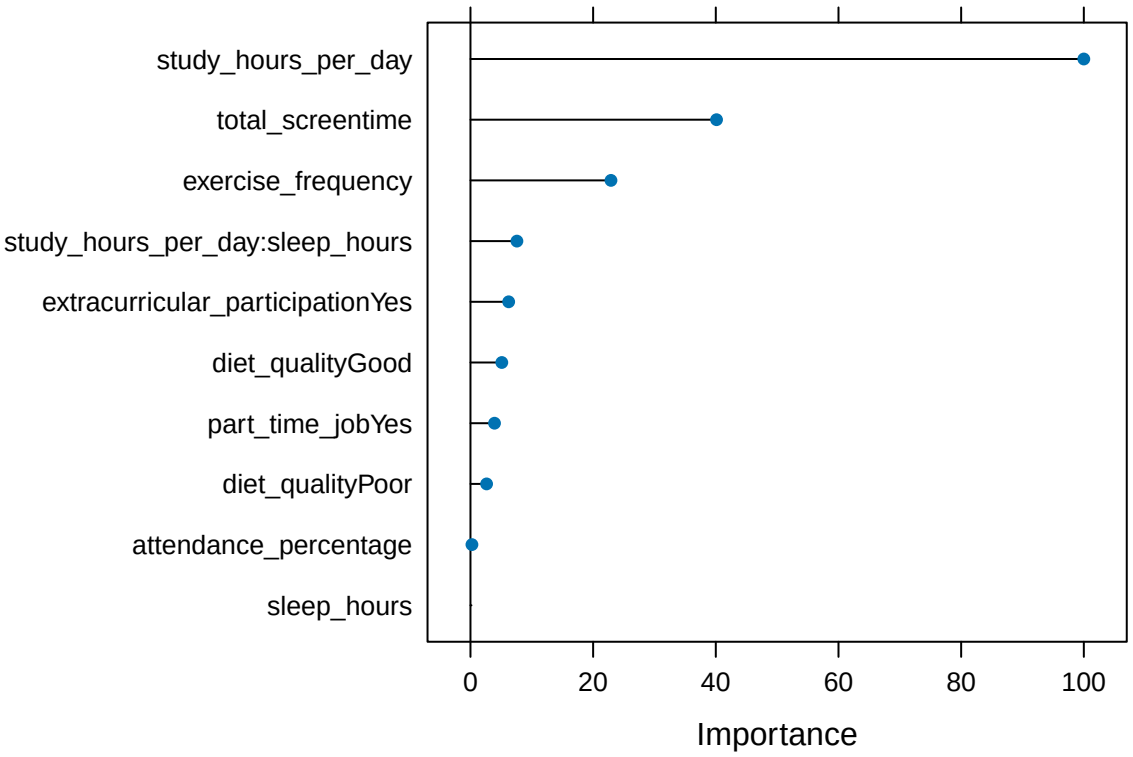

```
# 3. Predicting on test data
ridge_pred <- predict(cv_ridge_model, newdata = student_subset[test, ])

print(cv_ridge_model$bestTune)
```

```
##     alpha lambda
## 100     0      1
```

```r
# 4. Evaluate performance
ridge_actuals <- student_subset[test, ]$exam_score
ridge_performance <- postResample(pred = ridge_pred, obs = ridge_actuals)

print(ridge_performance)
```

```
##      RMSE  Rsquared       MAE
## 7.6933524 0.7856326 6.3060156
```

```r
# 5. Variable Importance
plot(varImp(cv_ridge_model))
```



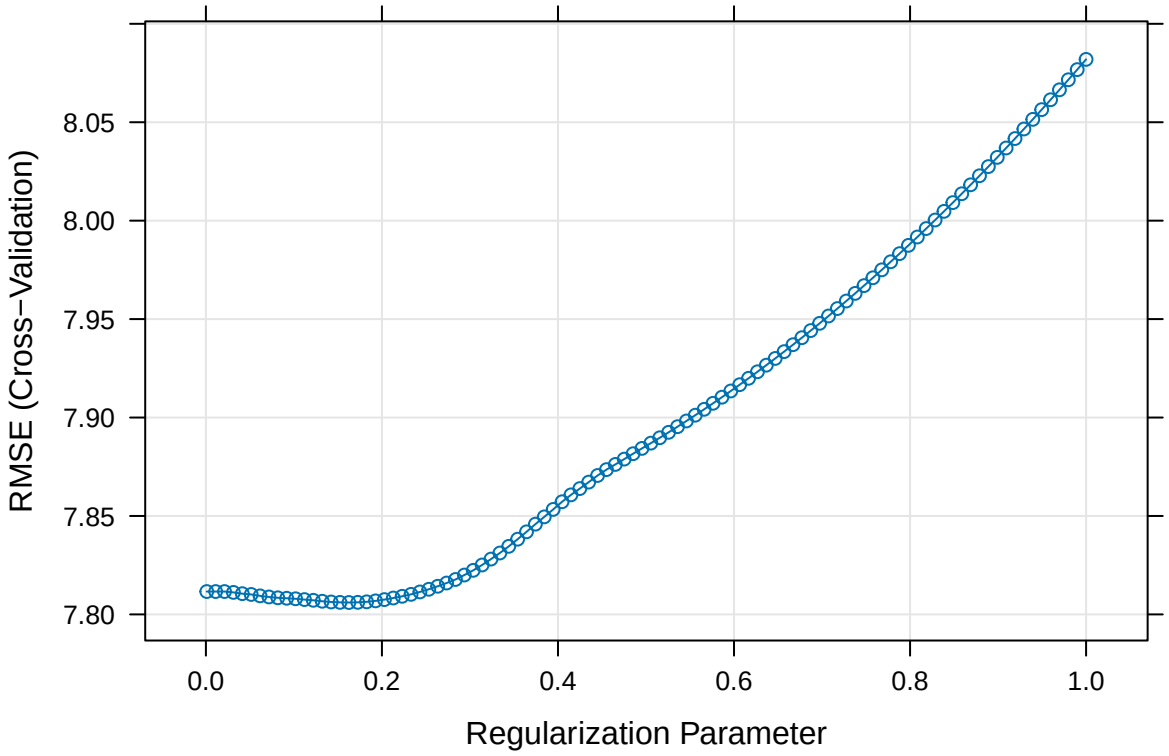
#Creating Lasso Model

```r
# 1. set tuning parameters
lasso_grid <- expand.grid(
  alpha = 1,                    # alpha = 0 → Ridge
  lambda = seq(0.001, 1, length = 100)  # lambda values to test
)

# 2. Train the model
set.seed(45)

cv_lasso_model <- train(
  exam_score ~ study_hours_per_day * sleep_hours + part_time_job +
```

```
    total_screentime + attendance_percentage + exercise_frequency +
    diet_quality + extracurricular_participation,
  data = student_subset[train, ],
  method = "glmnet",
  trControl = cv_control,
  tuneGrid = lasso_grid
)

# Get the best lambda chosen by cross-validation
lasso_best_lambda <- cv_lasso_model$bestTune$lambda

# Get lasso model coefficients
lasso_coefs <- coef(cv_lasso_model$finalModel, s = lasso_best_lambda)
print(lasso_coefs)
```

```
## 11 x 1 sparse Matrix of class "dgCMatrix"
##                                          s1
## (Intercept)                    2.407441e+01
## study_hours_per_day            8.310510e+00
## sleep_hours                    1.250769e+00
## part_time_jobYes               9.066938e-06
## total_screentime              -2.405057e+00
## attendance_percentage          1.206924e-01
## exercise_frequency             1.418486e+00
## diet_qualityGood              -5.859021e-02
## diet_qualityPoor               .
## extracurricular_participationYes  9.982504e-02
## study_hours_per_day:sleep_hours   1.644730e-01
```

```
plot(cv_lasso_model)
```



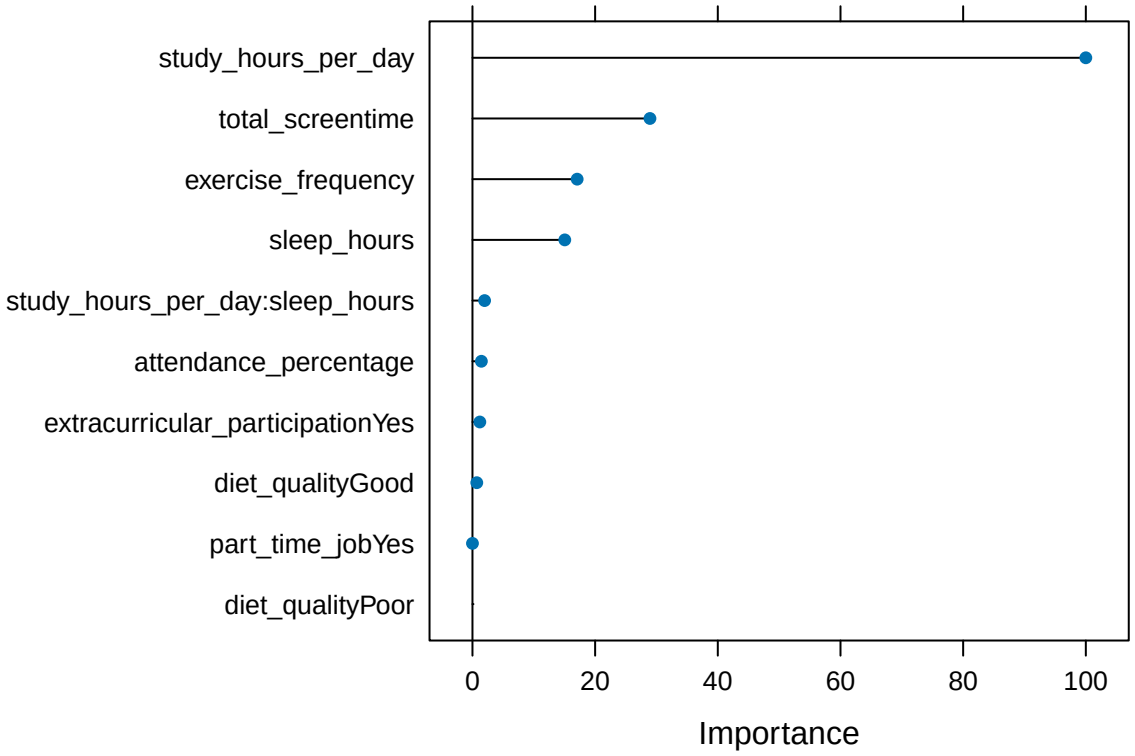

```r
# 3. Predicting on test data
lasso_pred <- predict(cv_lasso_model, newdata = student_subset[test, ])

print(cv_lasso_model$bestTune)
```

```
##    alpha    lambda
## 17     1 0.1624545
```

```r
# 4. Evaluate performance
lasso_actuals <- student_subset[test, ]$exam_score
lasso_performance <- postResample(pred = lasso_pred, obs = lasso_actuals)

print(lasso_performance)
```

```
##      RMSE  Rsquared       MAE
## 7.5565987 0.7915998 6.1878327
```

```r
# 5. Variable Importance
plot(varImp(cv_lasso_model))
```



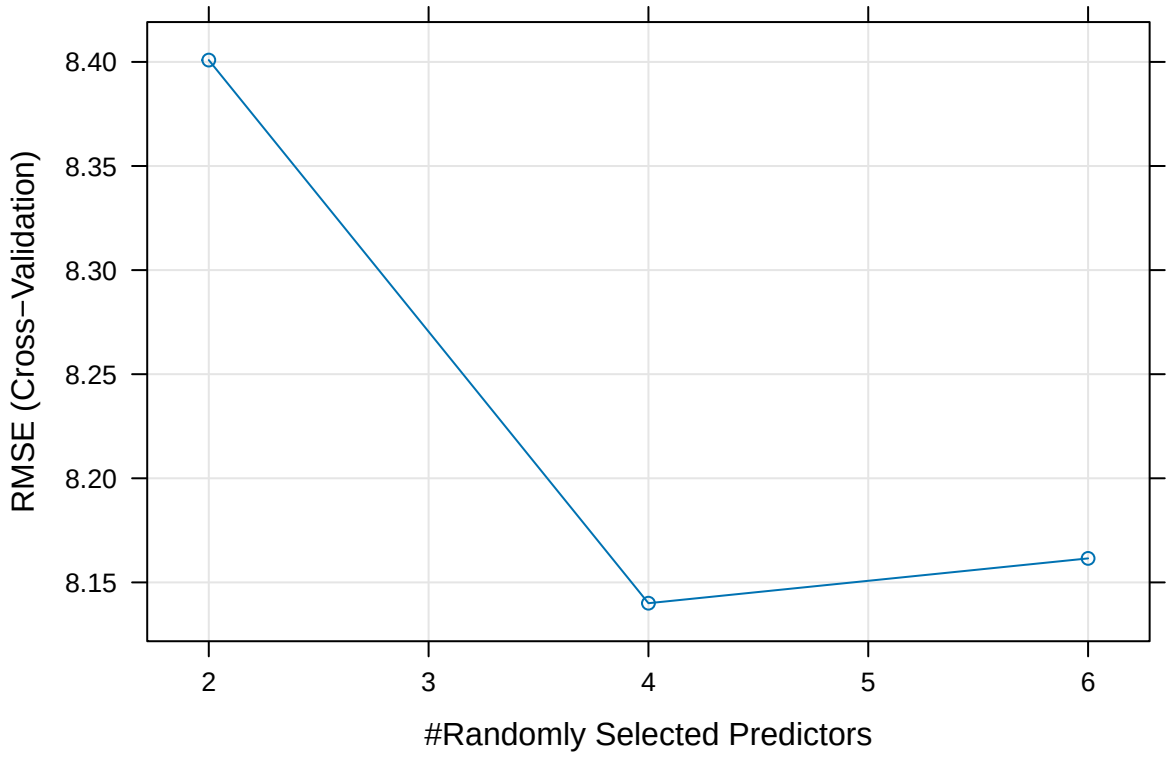
#Creating random forest Model

```
# 1. set tuning parameters
rf_grid <- expand.grid(mtry = c(2, 4, 6))  # Adjust values as needed

# 2. Train the model
set.seed(45)

cv_rf_model <- train(
  exam_score ~ study_hours_per_day * sleep_hours + part_time_job +
    total_screentime + attendance_percentage + exercise_frequency +
    diet_quality + extracurricular_participation,
  data = student_subset[train, ],
  method = "rf",
  trControl = cv_control,
  tuneGrid = rf_grid,
  importance = T
)
varImp(cv_rf_model) #variable importance for rf model
```

```
## rf variable importance
##
##                                 Overall
## total_screentime                100.000
## study_hours_per_day              82.073
## study_hours_per_day:sleep_hours  79.691
```

```
## exercise_frequency                62.186
## sleep_hours                       29.555
## attendance_percentage             14.779
## part_time_jobYes                   8.227
## diet_qualityGood                   5.743
## diet_qualityPoor                   3.020
## extracurricular_participationYes   0.000
```

```
plot(cv_rf_model)
```




```
# 3. Predicting on test data
rf_pred <- predict(cv_rf_model, newdata = student_subset[test, ])

print(cv_rf_model$bestTune)
```

```
##   mtry
## 2    4
```

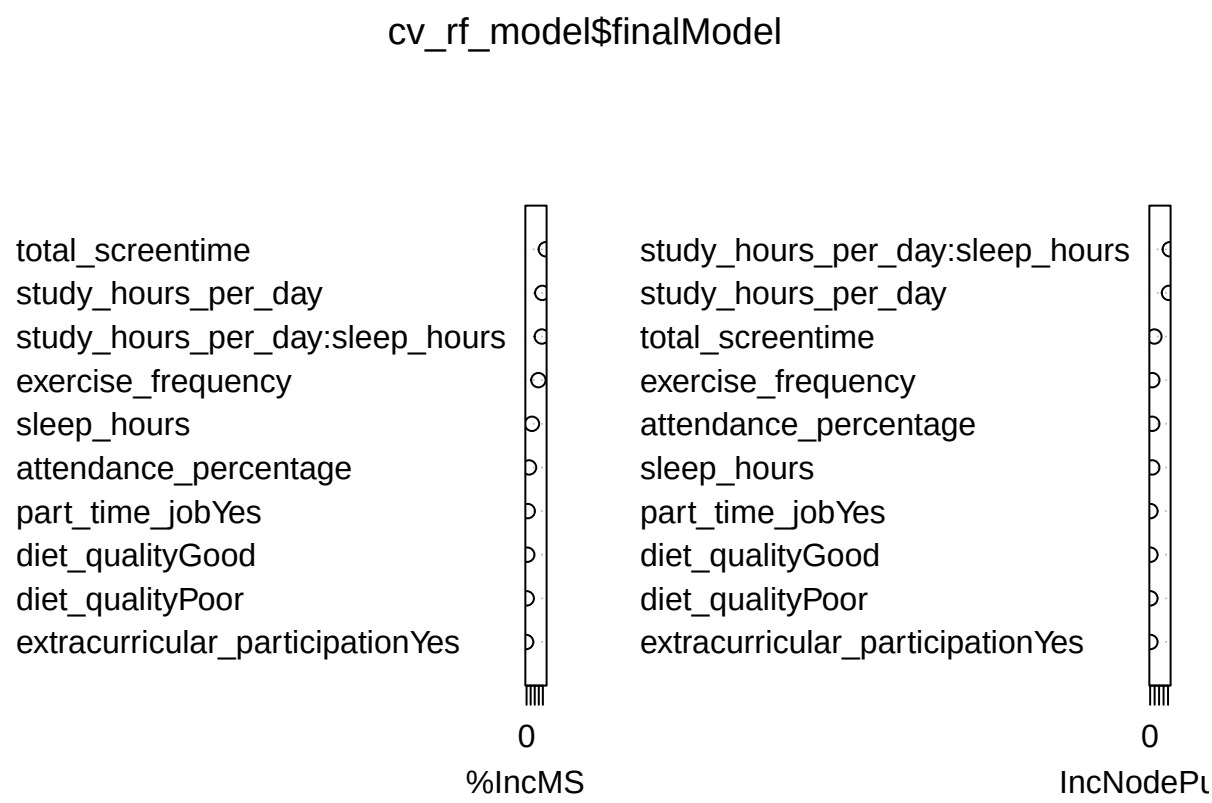
```
# 4. Evaluate performance
rf_actuals <- student_subset[test, ]$exam_score
rf_performance <- postResample(pred = rf_pred, obs = rf_actuals)

print(rf_performance)
```

```
##      RMSE  Rsquared       MAE
## 7.8234067 0.7779339 6.3691204
```

```r
# 5. Variable Importance for performance
varImp(cv_rf_model$finalModel)
```

```
##                                    Overall
## study_hours_per_day              38.569443
## sleep_hours                      13.201869
## part_time_jobYes                  2.899911
## total_screentime                 47.228542
## attendance_percentage             6.065111
## exercise_frequency               28.963738
## diet_qualityGood                  1.700277
## diet_qualityPoor                  0.385009
## extracurricular_participationYes -1.073701
## study_hours_per_day:sleep_hours  37.418741
```

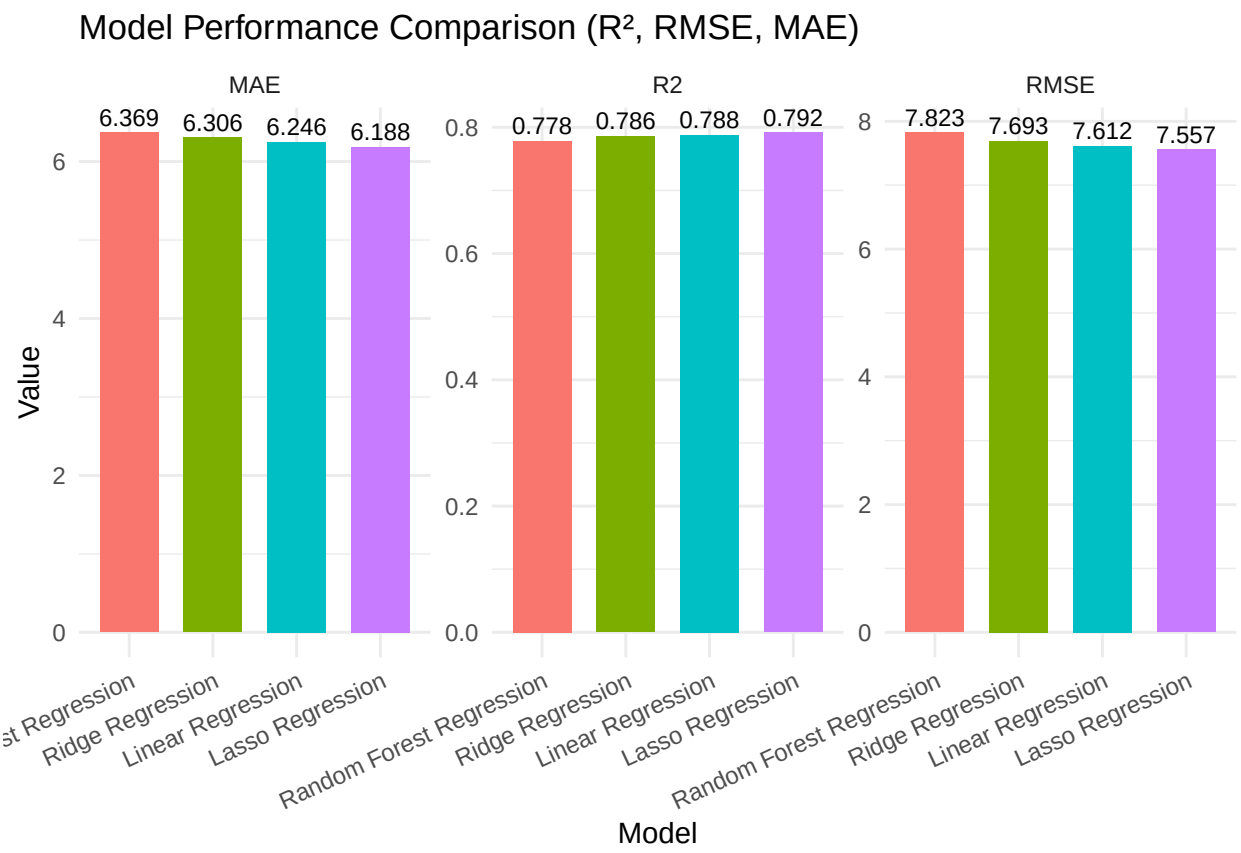
```r
varImpPlot(cv_rf_model$finalModel)
```



#Creating data frame of complied metrics

```r
model_metrics_vals <- data.frame(
  model = c("Linear Regression", "Ridge Regression", "Lasso Regression","Random Forest Regression"),
  R2 = c(lm_r2, as.numeric(ridge_performance[2]), as.numeric(lasso_performance[2]),
         as.numeric(rf_performance[2])),
  RMSE = c(lm_rmse, as.numeric(ridge_performance[1]), as.numeric(lasso_performance[1]),
           as.numeric(rf_performance[1])),
```

```
  MAE = c(lm_mae, as.numeric(ridge_performance[3]), as.numeric(lasso_performance[3]),
          as.numeric(rf_performance[3]))
)

View(model_metrics_vals)



metrics_long <- pivot_longer(model_metrics_vals,
                             cols = c("R2", "RMSE", "MAE"),
                             names_to = "Metric",
                             values_to = "Value")


metrics_long <- metrics_long %>%
  group_by(Metric) %>%
  mutate(model = fct_reorder(model,
                             ifelse(Metric == "R2", Value, -Value))) %>%
  ungroup()

# Plot
ggplot(metrics_long, aes(x = model, y = Value, fill = model)) +
  geom_bar(stat = "identity", width = 0.7) +
  geom_text(aes(label = round(Value, 3)), vjust = -0.4, size = 3) +
  facet_wrap(~ Metric, scales = "free_y") +
  labs(title = "Model Performance Comparison (R², RMSE, MAE)",
       x = "Model", y = "Value") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 25, hjust = 1),
        legend.position = "none")
```



Classifying students that have passed an exam by a certain score (>= 70)

```
#Creating new data frame. New column of pass and fail, dropping exam score and id.
student_features <- student_factor %>%
  mutate(pass = ifelse(exam_score >= 70, "Pass", "Fail"),
         pass = as.factor(pass)) %>%
  select(-exam_score)


#Splitting data to 80/20
log_train = sample(c(TRUE, FALSE), size = nrow(student_features), prob = c(0.8, 0.2), replace = TRUE)
log_test=student_features[!log_train,]
dim(log_test)
```

```
## [1] 214  16
```

```
#Convert the log_train to training dataframe along with test.
train_data <- student_features[log_train, ]
test_data <- student_features[!log_train, ]

logit_model <- glm(pass ~ ., data = train_data, family = binomial)
```

```r
summary(logit_model)
```

```
##
## Call:
## glm(formula = pass ~ ., family = binomial, data = train_data)
##
## Deviance Residuals:
##      Min        1Q    Median        3Q       Max
## -2.92699  -0.16959  -0.00044   0.19995   2.63592
##
## Coefficients: (1 not defined because of singularities)
##                                   Estimate Std. Error z value Pr(>|z|)
## (Intercept)                     -25.51048    2.93962  -8.678  < 2e-16 ***
## age                               0.04513    0.06307   0.715   0.4743
## genderMale                        0.15632    0.29489   0.530   0.5960
## genderOther                      -0.41747    0.62146  -0.672   0.5017
## study_hours_per_day               3.62860    0.31853  11.392  < 2e-16 ***
## social_media_hours               -0.98999    0.15155  -6.533 6.46e-11 ***
## netflix_hours                    -1.04089    0.15914  -6.541 6.12e-11 ***
## part_time_jobYes                  0.19782    0.34819   0.568   0.5699
## attendance_percentage             0.07479    0.01579   4.736 2.18e-06 ***
## sleep_hours                       0.75589    0.12739   5.933 2.97e-09 ***
## diet_qualityGood                 -0.28823    0.32030  -0.900   0.3682
## diet_qualityPoor                 -0.49766    0.39075  -1.274   0.2028
## exercise_frequency                0.56361    0.08594   6.558 5.45e-11 ***
## parental_education_levelHigh School  -0.31004    0.32693  -0.948   0.3430
## parental_education_levelMaster   -0.50644    0.43232  -1.171   0.2414
## parental_education_levelNone     -1.00205    0.55025  -1.821   0.0686 .
## internet_qualityGood             -0.34009    0.31936  -1.065   0.2869
## internet_qualityPoor             -0.21462    0.42013  -0.511   0.6095
## mental_health_rating              0.72911    0.07591   9.605  < 2e-16 ***
## extracurricular_participationYes -0.10319    0.30126  -0.343   0.7319
## total_screentime                      NA         NA      NA       NA
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1089.61  on 785  degrees of freedom
## Residual deviance:  328.98  on 766  degrees of freedom
## AIC: 368.98
##
## Number of Fisher Scoring iterations: 7
```

```r
test_probs <- predict(logit_model, newdata = test_data, type = "response")
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading
```

```r
pred_pass <- ifelse(test_probs >= 0.7, "Pass", "Fail")
pred_pass <- as.factor(pred_pass)
pred_pass <- factor(pred_pass, levels = levels(test_data$pass))
```

#Confusion Matrix

```
confusionMatrix(pred_pass, test_data$pass)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Fail Pass
##       Fail   89   18
##       Pass    5  102
##
##                Accuracy : 0.8925
##                  95% CI : (0.8431, 0.9306)
##     No Information Rate : 0.5607
##     P-Value [Acc > NIR] : < 2e-16
##
##                   Kappa : 0.785
##
##  Mcnemar's Test P-Value : 0.01234
##
##             Sensitivity : 0.9468
##             Specificity : 0.8500
##          Pos Pred Value : 0.8318
##          Neg Pred Value : 0.9533
##              Prevalence : 0.4393
##          Detection Rate : 0.4159
##    Detection Prevalence : 0.5000
##       Balanced Accuracy : 0.8984
##
##        'Positive' Class : Fail
##
```

#Creating stepwise model

```
#Labels for pass

true_labels_numeric <- ifelse(test_data$pass == "Pass", 1, 0)


#Step wise Selection
step_model <- step(logit_model, direction = "both")
```

```
## Start:  AIC=368.98
## pass ~ age + gender + study_hours_per_day + social_media_hours +
##     netflix_hours + part_time_job + attendance_percentage + sleep_hours +
##     diet_quality + exercise_frequency + parental_education_level +
##     internet_quality + mental_health_rating + extracurricular_participation +
##     total_screentime
##
##
## Step:  AIC=368.98
## pass ~ age + gender + study_hours_per_day + social_media_hours +
##     netflix_hours + part_time_job + attendance_percentage + sleep_hours +
```

```
##     diet_quality + exercise_frequency + parental_education_level +
##     internet_quality + mental_health_rating + extracurricular_participation
##
##                                 Df Deviance    AIC
## - gender                         2   329.90 365.90
## - internet_quality               2   330.12 366.12
## - diet_quality                   2   330.78 366.78
## - parental_education_level       3   332.88 366.88
## - extracurricular_participation  1   329.09 367.09
## - part_time_job                  1   329.30 367.30
## - age                            1   329.49 367.49
## <none>                               328.98 368.98
## - attendance_percentage          1   354.05 392.05
## - sleep_hours                    1   371.62 409.62
## - netflix_hours                  1   382.01 420.01
## - exercise_frequency             1   383.88 421.88
## - social_media_hours             1   384.18 422.18
## - mental_health_rating           1   504.30 542.30
## - study_hours_per_day            1   958.28 996.28
##
## Step:  AIC=365.9
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     part_time_job + attendance_percentage + sleep_hours + diet_quality +
##     exercise_frequency + parental_education_level + internet_quality +
##     mental_health_rating + extracurricular_participation
##
##                                 Df Deviance    AIC
## - internet_quality               2   331.32 363.32
## - parental_education_level       3   333.82 363.82
## - diet_quality                   2   331.94 363.94
## - extracurricular_participation  1   330.00 364.00
## - part_time_job                  1   330.17 364.17
## - age                            1   330.51 364.51
## <none>                               329.90 365.90
## + gender                         2   328.98 368.98
## - attendance_percentage          1   355.36 389.36
## - sleep_hours                    1   371.92 405.92
## - netflix_hours                  1   384.13 418.13
## - social_media_hours             1   384.66 418.66
## - exercise_frequency             1   385.19 419.19
## - mental_health_rating           1   504.77 538.77
## - study_hours_per_day            1   958.69 992.69
##
## Step:  AIC=363.32
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     part_time_job + attendance_percentage + sleep_hours + diet_quality +
##     exercise_frequency + parental_education_level + mental_health_rating +
##     extracurricular_participation
##
##                                 Df Deviance    AIC
## - diet_quality                   2   333.13 361.13
## - extracurricular_participation  1   331.41 361.41
## - parental_education_level       3   335.59 361.59
## - part_time_job                  1   331.62 361.62
```

```
## - age                             1   331.93 361.93
## <none>                                331.32 363.32
## + internet_quality                2   329.90 365.90
## + gender                          2   330.12 366.12
## - attendance_percentage           1   357.02 387.02
## - sleep_hours                     1   374.07 404.07
## - social_media_hours              1   386.60 416.60
## - netflix_hours                   1   387.39 417.39
## - exercise_frequency              1   388.70 418.70
## - mental_health_rating            1   507.30 537.30
## - study_hours_per_day             1   961.78 991.78
##
## Step:  AIC=361.13
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     part_time_job + attendance_percentage + sleep_hours + exercise_frequency +
##     parental_education_level + mental_health_rating + extracurricular_participation
##
##                                 Df Deviance    AIC
## - extracurricular_participation  1   333.28 359.28
## - parental_education_level       3   337.33 359.33
## - part_time_job                  1   333.37 359.37
## - age                            1   333.79 359.79
## <none>                                333.13 361.13
## + diet_quality                   2   331.32 363.32
## + gender                         2   331.71 363.71
## + internet_quality               2   331.94 363.94
## - attendance_percentage          1   358.40 384.40
## - sleep_hours                    1   375.65 401.65
## - social_media_hours             1   389.05 415.05
## - netflix_hours                  1   390.05 416.05
## - exercise_frequency             1   391.69 417.69
## - mental_health_rating           1   507.55 533.55
## - study_hours_per_day            1   965.31 991.31
##
## Step:  AIC=359.28
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     part_time_job + attendance_percentage + sleep_hours + exercise_frequency +
##     parental_education_level + mental_health_rating
##
##                                 Df Deviance    AIC
## - parental_education_level       3   337.39 357.39
## - part_time_job                  1   333.50 357.50
## - age                            1   333.93 357.93
## <none>                                333.28 359.28
## + extracurricular_participation  1   333.13 361.13
## + diet_quality                   2   331.41 361.41
## + gender                         2   331.89 361.89
## + internet_quality               2   332.10 362.10
## - attendance_percentage          1   358.51 382.51
## - sleep_hours                    1   375.75 399.75
## - social_media_hours             1   389.06 413.06
## - netflix_hours                  1   390.16 414.16
## - exercise_frequency             1   391.96 415.96
## - mental_health_rating           1   507.57 531.57
```

```
## - study_hours_per_day            1   965.36 989.36
##
## Step:  AIC=357.39
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     part_time_job + attendance_percentage + sleep_hours + exercise_frequency +
##     mental_health_rating
##
##                                 Df Deviance    AIC
## - part_time_job                  1   337.47 355.47
## - age                            1   338.09 356.09
## <none>                               337.39 357.39
## + parental_education_level       3   333.28 359.28
## + extracurricular_participation  1   337.33 359.33
## + diet_quality                   2   335.62 359.62
## + internet_quality               2   335.88 359.88
## + gender                         2   335.90 359.90
## - attendance_percentage          1   361.08 379.08
## - sleep_hours                    1   379.42 397.42
## - netflix_hours                  1   391.88 409.88
## - social_media_hours             1   392.78 410.78
## - exercise_frequency             1   396.15 414.15
## - mental_health_rating           1   514.20 532.20
## - study_hours_per_day            1   966.13 984.13
##
## Step:  AIC=355.47
## pass ~ age + study_hours_per_day + social_media_hours + netflix_hours +
##     attendance_percentage + sleep_hours + exercise_frequency +
##     mental_health_rating
##
##                                 Df Deviance    AIC
## - age                            1   338.19 354.19
## <none>                               337.47 355.47
## + part_time_job                  1   337.39 357.39
## + extracurricular_participation  1   337.41 357.41
## + parental_education_level       3   333.50 357.50
## + diet_quality                   2   335.74 357.74
## + internet_quality               2   335.94 357.94
## + gender                         2   336.01 358.01
## - attendance_percentage          1   361.09 377.09
## - sleep_hours                    1   379.56 395.56
## - netflix_hours                  1   391.91 407.91
## - social_media_hours             1   392.78 408.78
## - exercise_frequency             1   396.15 412.15
## - mental_health_rating           1   514.22 530.22
## - study_hours_per_day            1   966.37 982.37
##
## Step:  AIC=354.19
## pass ~ study_hours_per_day + social_media_hours + netflix_hours +
##     attendance_percentage + sleep_hours + exercise_frequency +
##     mental_health_rating
##
##                                 Df Deviance    AIC
## <none>                               338.19 354.19
## + age                            1   337.47 355.47
```

```
## + part_time_job                  1   338.09 356.09
## + extracurricular_participation  1   338.14 356.14
## + parental_education_level       3   334.20 356.20
## + diet_quality                   2   336.41 356.41
## + gender                         2   336.60 356.60
## + internet_quality               2   336.65 356.65
## - attendance_percentage          1   361.27 375.27
## - sleep_hours                    1   380.57 394.57
## - netflix_hours                  1   391.94 405.94
## - social_media_hours             1   393.16 407.16
## - exercise_frequency             1   396.54 410.54
## - mental_health_rating           1   514.22 528.22
## - study_hours_per_day            1   966.43 980.43
```

```r
# Predict on test set
step_test_probs <- predict(step_model, newdata = test_data, type = "response")
step_pred_pass <- ifelse(step_test_probs >= 0.7, "Pass", "Fail")
step_pred_pass <- factor(step_pred_pass, levels = levels(test_data$pass))

head(step_pred_pass)
```

```
##    1    2    8   17   19   22
## Fail Pass Pass Fail Fail Pass
## Levels: Fail Pass
```

```r
summary(step_model)
```

```
##
## Call:
## glm(formula = pass ~ study_hours_per_day + social_media_hours +
##     netflix_hours + attendance_percentage + sleep_hours + exercise_frequency +
##     mental_health_rating, family = binomial, data = train_data)
##
## Deviance Residuals:
##      Min        1Q    Median        3Q       Max
## -2.51336  -0.19110  -0.00057   0.21591   2.59251
##
## Coefficients:
##                         Estimate Std. Error z value Pr(>|z|)
## (Intercept)           -24.16436    2.39911 -10.072  < 2e-16 ***
## study_hours_per_day     3.49986    0.29768  11.757  < 2e-16 ***
## social_media_hours     -0.96219    0.14677  -6.556 5.54e-11 ***
## netflix_hours          -0.99628    0.15080  -6.606 3.94e-11 ***
## attendance_percentage   0.06829    0.01492   4.578 4.69e-06 ***
## sleep_hours             0.74653    0.12581   5.934 2.96e-09 ***
## exercise_frequency      0.56897    0.08430   6.749 1.49e-11 ***
## mental_health_rating    0.70055    0.07145   9.804  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 1089.61  on 785  degrees of freedom
```

```
## Residual deviance:  338.19  on 778  degrees of freedom
## AIC: 354.19
##
## Number of Fisher Scoring iterations: 7
```

```
#create confusion matrix
confusionMatrix(step_pred_pass, test_data$pass)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Fail Pass
##       Fail   89   19
##       Pass    5  101
##
##                Accuracy : 0.8879
##                  95% CI : (0.8377, 0.9268)
##     No Information Rate : 0.5607
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.776
##
##  Mcnemar's Test P-Value : 0.007963
##
##             Sensitivity : 0.9468
##             Specificity : 0.8417
##          Pos Pred Value : 0.8241
##          Neg Pred Value : 0.9528
##              Prevalence : 0.4393
##          Detection Rate : 0.4159
##    Detection Prevalence : 0.5047
##       Balanced Accuracy : 0.8942
##
##        'Positive' Class : Fail
##
```

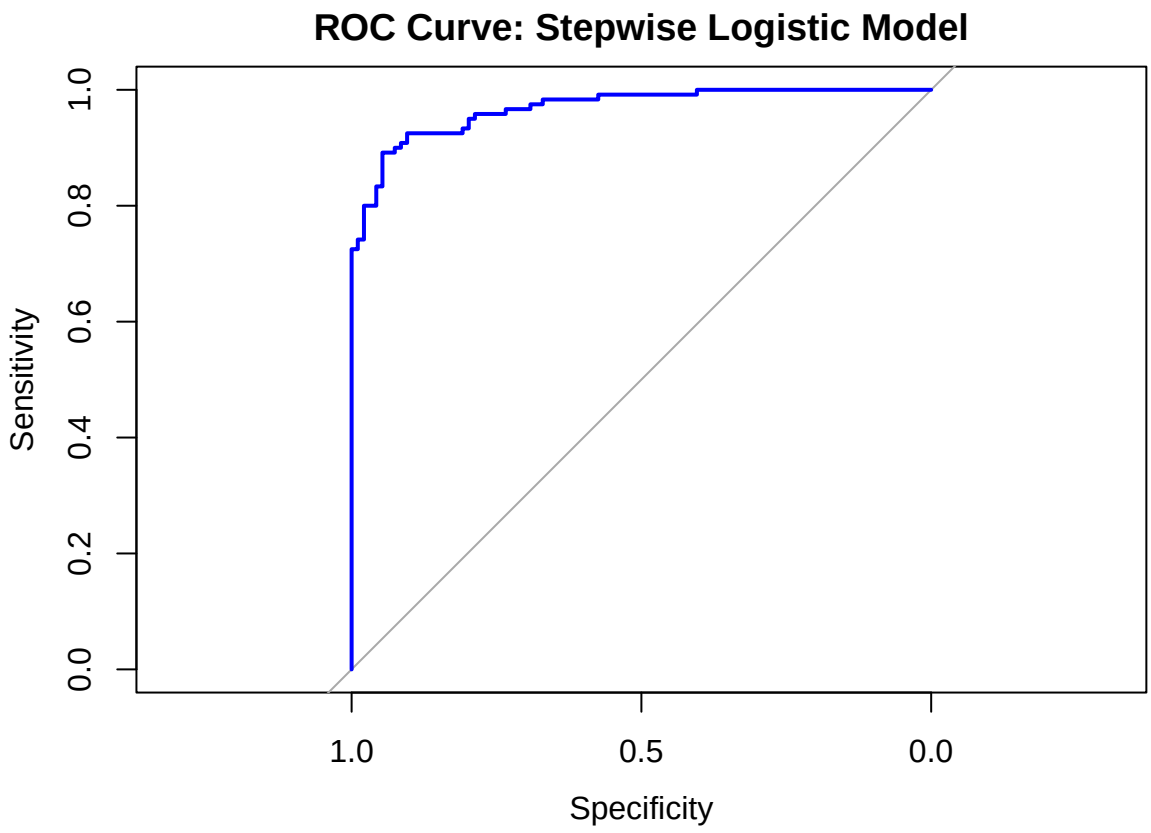
#ROC Curve

```
#Labels for pass
true_labels_numeric <- ifelse(test_data$pass == "Pass", 1, 0)

# Create ROC object
roc_obj <- roc(response = true_labels_numeric, predictor = step_test_probs)
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
par(mfrow = c(1, 1))
# Plotting ROC curve
plot(roc_obj, col = "blue", lwd = 2, main = "ROC Curve: Stepwise Logistic Model")
```




```
# AUC score
auc(roc_obj)
```

```
## Area under the curve: 0.9684
```

#Cross validation

```
#Cross Validating Logistic Regression Model

train_control <- trainControl(method = "cv", number = 10, classProbs = TRUE,
  summaryFunction = twoClassSummary, savePredictions = "final"
)

cv_logit_model <- train(pass ~ ., data = student_features, method = "glm",
  family = "binomial", trControl = train_control, metric = "Accuracy"
)
```

```
## Warning in train.default(x, y, weights = w, ...): The metric "Accuracy" was not
## in the result set. ROC will be used instead.
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading
```

```
## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
```

```
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
## prediction from a rank-deficient fit may be misleading

## Warning in predict.lm(object, newdata, se.fit, scale = 1, type = if (type == :
```

```
## prediction from a rank-deficient fit may be misleading
```

```
print(cv_logit_model)
```

```
## Generalized Linear Model
##
## 1000 samples
##   15 predictor
##    2 classes: 'Fail', 'Pass'
##
## No pre-processing
## Resampling: Cross-Validated (10 fold)
## Summary of sample sizes: 900, 900, 900, 900, 900, 900, ...
## Resampling results:
##
##   ROC        Sens       Spec
##   0.9672364  0.8957908  0.9020739
```

Cross valdiation predictions

```
#Cross Validation Predictions
cv_predictions <- cv_logit_model$pred
confusionMatrix(data = cv_predictions$pred, reference = cv_predictions$obs)
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction Fail Pass
##       Fail  438   50
##       Pass   51  461
##
##                Accuracy : 0.899
##                  95% CI : (0.8786, 0.917)
##     No Information Rate : 0.511
##     P-Value [Acc > NIR] : <2e-16
##
##                   Kappa : 0.7979
##
##  Mcnemar's Test P-Value : 1
##
##             Sensitivity : 0.8957
##             Specificity : 0.9022
##          Pos Pred Value : 0.8975
##          Neg Pred Value : 0.9004
##              Prevalence : 0.4890
##          Detection Rate : 0.4380
##    Detection Prevalence : 0.4880
##       Balanced Accuracy : 0.8989
##
##        'Positive' Class : Fail
##
```

#ANOVA model

```r
#Ho: The full model is more efficient
#Ha: The stepwise model is more efficient

#Full model vs step wise model
anova(step_model, logit_model, test = "Chisq")
```

```
## Analysis of Deviance Table
##
## Model 1: pass ~ study_hours_per_day + social_media_hours + netflix_hours +
##     attendance_percentage + sleep_hours + exercise_frequency +
##     mental_health_rating
## Model 2: pass ~ age + gender + study_hours_per_day + social_media_hours +
##     netflix_hours + part_time_job + attendance_percentage + sleep_hours +
##     diet_quality + exercise_frequency + parental_education_level +
##     internet_quality + mental_health_rating + extracurricular_participation +
##     total_screentime
##   Resid. Df Resid. Dev Df Deviance Pr(>Chi)
## 1       778     338.19
## 2       766     328.98 12   9.2169   0.6843
```